

Faculty of Media Engineering and Technology
Computer Systems Architecture – CSEN601

Project Report

Package 3: Double Big Harvard Combo Large Arithmetic Shifts

Spring 2026

Submitted: Saturday, 16 May 2026

1. Project Overview

This report documents the design and implementation of a simulated processor for CSEN601 Package 3: Double Big Harvard Combo Large Arithmetic Shifts. The simulator is written in C and models a Harvard-architecture CPU with a 3-stage pipeline (Instruction Fetch, Instruction Decode, Execute).

The simulator faithfully reproduces the specified memory organisation, register file, instruction set architecture, pipeline timing, hazard detection and forwarding, and all required console printouts.

1.1 Architecture Summary

Component	Specification
Architecture	Harvard (separate instruction and data memories)
Instruction Memory	1024 x 16-bit words (word-addressable)
Data Memory	2048 x 8-bit bytes (byte/word-addressable)
Registers	64 x 8-bit GPRs (R0–R63) + 8-bit SREG + 16-bit PC
Instruction Size	16 bits (2 bytes)
Instruction Formats	R-Format [OPCODE 4 R1 6 R2 6] and I-Format [OPCODE 4 R1 6 IMM 6]
Instruction Count	12 instructions (opcodes 0–11)
Pipeline Stages	3 stages: IF → ID → EX
Pipeline Depth	Up to 3 instructions in parallel
Clock Cycle Formula	$3 + (n - 1) \times 1$, where n = number of instructions
Status Register	5 flags: C (Carry), V (Overflow), N (Negative), S (Sign), Z (Zero)

1.2 Instruction Set

Opcode	Mnemonic	Format	Operation	Flags Updated
0	ADD	R	$R1 = R1 + R2$	C, V, N, S, Z
1	SUB	R	$R1 = R1 - R2$	V, N, S, Z
2	MUL	R	$R1 = R1 * R2$ (8-bit trunc.)	N, Z
3	MOVI	I	$R1 = IMM$	none
4	BEQZ	I	if $(R1 == 0)$ $PC = PC + 1 + IMM$	none
5	ANDI	I	$R1 = R1 \& IMM$	N, Z
6	EOR	R	$R1 = R1 \text{ XOR } R2$	N, Z
7	BR	R	$PC = R1 \parallel R2$ (16-bit concat.)	none
8	SLC	I	$R1 = \text{rotate_left}(R1, IMM)$	N, Z

Opcode	Mnemonic	Format	Operation	Flags Updated
9	SRC	I	R1 = rotate_right(R1, IMM)	N, Z
10	LDR	I	R1 = MEM[ADDRESS]	none
11	STR	I	MEM[ADDRESS] = R1	none

2. Module Breakdown

The simulator is divided into five source modules, each with a matching header file, plus a top-level entry point. The modules are described below.

2.1 main.c — Entry Point

The main() function is the program entry point. It accepts a single command-line argument: the path to the assembly source file. It orchestrates initialisation and launches the pipeline simulation.

Signature: int main(int argc, char *argv[])

Input: argv[1] — path to the assembly text file

Output: Returns 0 on success, 1 on usage error or file-load failure

Execution sequence:

- Prints a startup banner identifying Package 3.
- Calls initMemory() to zero both memories and reset the PC.
- Calls initRegisters() to zero all GPRs and SREG.
- Calls loadProgram(filename) to parse and encode the assembly file into instruction memory.
- Calls initPipeline(numInstructions) to set up pipeline state.
- Calls runPipeline() to start the clock-cycle simulation loop.

2.2 memory.c / memory.h — Memory Subsystem

This module owns the two physical memories and the Program Counter. It provides all read/write operations and the assembly parser/loader.

2.2.1 Global State

- uint16_t instructionMemory[1024] — 16-bit instruction memory, word-addressable.
- int8_t dataMemory[2048] — 8-bit data memory, byte-addressable.
- uint16_t PC — 16-bit program counter.

2.2.2 Functions

void initMemory()

Zero-fills both memory arrays with memset() and resets PC to 0. Prints a confirmation line for each operation. Called once at program start by main().

void storeInstruction(uint16_t address, uint16_t encodedInstruction)

Input: address (0–1023), 16-bit encoded instruction

Bounds-checks address and stores the encoded instruction. Called by loadProgram() once per parsed line.

uint16_t readInstruction(uint16_t address)

Input: address (0–1023) **Output:** 16-bit instruction word

Returns the instruction stored at the given address. Called every clock cycle by stageFetch() in the IF stage.

int8_t readData(uint16_t address)

Input: address (0–2047) **Output:** signed 8-bit byte

Reads one byte from data memory. Used by the LDR instruction during the EX stage.

void writeData(uint16_t address, int8_t value)

Input: address (0–2047), signed 8-bit value

Writes a byte to data memory and immediately prints the mandatory log line: [EX] Data Memory Updated: MEM[n] = v. Used by the STR instruction during the EX stage.

uint16_t encodeRFormat(uint8_t opcode, uint8_t r1, uint8_t r2)

Input: 4-bit opcode, two 6-bit register numbers **Output:** 16-bit encoded word

Packs the three fields into a 16-bit integer: bits [15:12] = opcode, bits [11:6] = R1, bits [5:0] = R2. Bit-masks ensure no field overflows its allocation.

uint16_t encodeIFormat(uint8_t opcode, uint8_t r1, int8_t imm)

Input: 4-bit opcode, 6-bit register, signed 6-bit immediate **Output:** 16-bit encoded word

Same layout as R-Format except the lower 6 bits hold the immediate. The & 0x3F mask preserves the 2's complement bit pattern for negative values.

int loadProgram(const char *filename)

Input: path to assembly text file **Output:** number of instructions loaded, or -1 on file error

Main parser. Opens the file, reads lines one at a time, tokenises each line with sscanf, and looks the mnemonic up in a static 12-entry OpcodeEntry table. Register operands are parsed with parseRegister() (strips the leading 'R' and converts to integer). The immediate is read with atoi() and range-checked. The instruction is encoded and stored in instruction memory. A formatted table row is printed for each instruction. Blank lines and lines beginning with '#' or '/' are silently skipped.

void printInstructionMemory()

Dumps all 1024 instruction memory slots (address, hex, binary) after the simulation ends, satisfying the spec requirement to print the full memory content.

void printDataMemory()

Dumps all 2048 data memory slots (address, decimal, binary) at end of simulation.

2.3 registers.c / registers.h — Register File & Status Register

This module manages the 64 GPRs and the 5-flag SREG. It exposes typed accessors and flag-update helpers so that only the correct flags are touched for each instruction type.

2.3.1 Global State

- `int8_t GPR[64]` — 64 general-purpose 8-bit signed registers (R0–R63).
- `uint8_t SREG` — 8-bit status register; only bits [4:0] are used.

SREG bit layout:

Bit 7	Bit 6	Bit 5	Bit 4 (C)	Bit 3 (V)	Bit 2 (N)	Bit 1 (S)	Bit 0 (Z)
0 (reserved)	0 (reserved)	0 (reserved)	Carry	Overflow	Negative	Sign	Zero

2.3.2 Functions

void initRegisters(void)

Zero-fills GPR[0..63] and resets SREG to 0x00. Prints confirmation lines. Called once at startup.

int8_t getRegister(uint8_t regNum)

Input: register index 0–63 **Output:** signed 8-bit value

Bounds-checked read. Returns 0 and prints an error on out-of-range access. Note: unlike Packages 1 and 2, all registers including R0 are normal writable GPRs in Package 3.

void setRegister(uint8_t regNum, int8_t value, const char *stageName)

Input: register index, new value, pipeline stage name for logging

Bounds-checked write. Always prints: [`<stage>`] Register Updated: R<n> = <value> as required by the spec. The `stageName` parameter allows different pipeline stages to identify themselves in the log output.

void updateCarryFlag(int val1, int val2)

Checks whether bit 8 of the unsigned sum $(val1 \& 0xFF) + (val2 \& 0xFF)$ is 1. Implements the spec's "Carry Flag Check" section exactly. Updates only the C bit in SREG. Called by ADD only.

void updateOverflowFlag(int8_t val1, int8_t val2, int8_t result8, int isSubtraction)

For ADD ($isSubtraction=0$): overflow if both operands share the same sign but the result has the opposite sign. For SUB ($isSubtraction=1$): overflow if the operands have different signs and the result shares the subtrahend's sign. Updates only the V bit. Called by ADD and SUB.

void updateNegativeFlag(int8_t result8)

Sets N if $result8 < 0$. Clears N otherwise. Updates only the N bit. Called by ADD, SUB, MUL, ANDI, EOR, SLC, SRC.

void updateSignFlag(void)

Computes $S = N \text{ XOR } V$ from the current SREG bits. Must be called after both N and V have been updated in the same cycle. Called by ADD and SUB.

void updateZeroFlag(int8_t result8)

Sets Z if $result8 == 0$, clears otherwise. Called by ADD, SUB, MUL, ANDI, EOR, SLC, SRC.

void updateFlagsADD(int8_t val1, int8_t val2, int8_t result8)

Convenience bundle for ADD: calls `updateCarryFlag`, `updateOverflowFlag` ($isSubtraction=0$), `updateNegativeFlag`, `updateSignFlag`, `updateZeroFlag` in the correct order. Prints the full SREG state after updating.

void updateFlagsSUB(int8_t val1, int8_t val2, int8_t result8)

Convenience bundle for SUB: skips the carry update (not required for SUB), then calls `updateOverflowFlag` ($isSubtraction=1$), `updateNegativeFlag`, `updateSignFlag`, `updateZeroFlag`.

void updateFlagsNZ(int8_t result8)

Minimal bundle for MUL, ANDI, EOR, SLC, SRC: updates only N and Z.

void printRegisters(void)

Prints all 64 GPR values in a formatted table (decimal, hex, binary) at end of simulation.

void printSREG(void)

Prints SREG with individual flag breakdown and the computed $S = N \text{ XOR } V$ formula at end of simulation.

2.4 decoder.c / decoder.h — Instruction Decoder

The decoder translates a raw 16-bit instruction word into a structured Instruction record. It is called during the ID stage every cycle. Per the spec (point 7), the pipeline always decodes from the binary-encoded word stored in memory; no metadata is carried forward from the parsing stage.

2.4.1 Data Structure

The Instruction struct contains:

Field	Type	Description
opcode	int	Instruction opcode (0–11)
r1	int	Destination / first source register index
r2	int	Second source register index (R-Format only; 0 for I-Format)
immediate	int	Sign-extended immediate value (I-Format only; 0 for R-Format)
type	char	'R' or 'I'
mnemonic	char[8]	ASCII mnemonic string, e.g. "ADD"

2.4.2 Functions

Instruction decode(uint16_t instruction)

Input: 16-bit encoded instruction **Output:** populated Instruction struct

Extracts fields by masking and shifting:

- bits [15:12] → opcode (mask 0xF after shifting right 12)
- bits [11:6] → r1 (mask 0x3F after shifting right 6)
- bits [5:0] → r2 or immediate (mask 0x3F)

Format detection: opcodes 0, 1, 2, 6, 7 are R-Format; all others are I-Format. For I-Format instructions, r2 is set to 0 and the immediate is sign-extended from 6 bits using signExtend6() (except for opcodes 8, 9, 10, 11, which use unsigned immediates). The mnemonic is assigned from a static name array indexed by opcode.

static int signExtend6(int value)

Input: raw 6-bit value (0–63) **Output:** sign-extended 32-bit integer

If bit 5 of value is set, OR-fills the upper 26 bits to produce the correct negative two's complement representation for the full int type.

2.5 alu.c / alu.h — Arithmetic Logic Unit

The ALU module implements the computational core of the EX stage. It handles all arithmetic, logical, and shift operations, and computes branch target addresses.

2.5.1 Functions

int8_t execute(int opcode, int8_t val1, int8_t val2)

Input: opcode, two 8-bit signed operands **Output:** 8-bit signed result

A switch statement dispatches to the correct operation:

Opcode	Operation	Implementation Note
0 (ADD)	val1 + val2	Widens to int before adding; truncates back to int8_t. Calls updateFlagsADD.
1 (SUB)	val1 - val2	Same widening pattern. Calls updateFlagsSUB.
2 (MUL)	val1 * val2 (truncated to 8-bit)	16-bit intermediate product; lower 8 bits kept. Calls updateFlagsNZ.
5 (ANDI)	val1 & val2	Bitwise AND. Calls updateFlagsNZ.
6 (EOR)	val1 ^ val2	Bitwise XOR. Calls updateFlagsNZ.
8 (SLC)	rotate_left(val1, 1)	MSB wraps to LSB via uint8_t cast. Calls updateFlagsNZ.
9 (SRC)	rotate_right(val1, 1)	LSB wraps to MSB. Calls updateFlagsNZ.
3,4,7,10,11	N/A — not ALU operations	Returns 0; handled by stageExecute() directly.

Note: SLC and SRC each rotate by 1 bit per instruction execution. The IMM field encodes a shift amount, but the pipeline calls execute() once per cycle, so applying the rotation once per opcode invocation is correct for a single-cycle EX stage.

uint16_t calculateBranch(uint16_t currentPC, int8_t immediate)

Input: current PC value, sign-extended 6-bit immediate **Output:** new 16-bit PC value

Implements the spec formula: $PC_{new} = currentPC + 1 + immediate$. The "+1" accounts for the PC increment that occurred during the fetch stage. Used by BEQZ when the condition is true, and not called for BR (which uses register concatenation instead).

int testALU(void)

Standalone unit test suite for the ALU (not invoked during normal simulation). Covers 6 test groups: arithmetic operations, logical operations, circular shifts, branch calculation, flag updates, and edge cases. Prints PASS/FAIL for each of the 29 individual test assertions and a summary at the end.

2.6 pipeline.c / pipeline.h — Pipeline Controller

This module owns the pipeline registers, stage functions, hazard detection, forwarding logic, and the simulation loop. It implements Package 3's 3-stage pipeline: IF → ID → EX.

2.6.1 Pipeline Registers (Latch Structs)

IF_ID_Register

Carries data from the IF stage to the ID stage. Fields: instruction (raw 16-bit word), pc (PC value at fetch time), valid (1 = contains a live instruction, 0 = bubble).

ID_EX_Register

Carries data from the ID stage to the EX stage. Fields: decoded (Instruction struct), val1 / val2 (register values read during decode, or forwarded values), pc (fetch-time PC, needed for branch address calculation), valid flag.

2.6.2 Static Pipeline State

- forwardValue / forwardReg / forwardValid — forwarding path: the result written by EX is held here so it can be injected into the next ID stage.
- branchTaken — set to 1 when a branch or BR executes; triggers pipeline flush.
- fetchedCount — tracks how many instructions have been fetched to know when to stop.
- lastExecuted* fields — snapshot of the last EX stage for logState() reporting.

2.6.3 Functions

void initPipeline(int numInstructions)

Input: total number of instructions in the program

Zeros all pipeline latches, resets counters (clockCycle, completedInstructions, fetchedCount), and clears the forwarding state. Prints a startup message. Called once from main() after loading the program.

void runPipeline(void)

The main simulation loop. Calls runCycle() repeatedly until all instructions have been fetched AND both pipeline latches are empty (no bubbles remain). After the loop, prints the final statistics banner, then calls printRegisters(), printSREG(), and the memory dumps.

void runCycle(void)

Executes one clock cycle. Saves the IF/ID latch before dispatch. Calls stageExecute() first (so forwarding values are ready before stageDecode reads them). If a branch was not taken, restores the saved IF/ID latch and calls stageDecode() followed by stageFetch(). If a branch was taken, prints cancellation messages for the ID and IF slots. Finally calls logState() and increments clockCycle.

void stageFetch(void)

IF stage. Reads the instruction at PC using `readInstruction()`, stores it and the current PC value into `IF_ID`, increments PC, and increments `fetchCount`. If `fetchCount` has already reached `totalInstructions`, sets `IF_ID.valid = 0` (bubble) and returns.

void stageDecode(void)

ID stage. If `IF_ID` is invalid (bubble), sets `ID_EX.valid = 0` and returns. Otherwise calls `decode()` on the raw instruction word, reads `val1` from `GPR[r1]`, and reads `val2` from `GPR[r2]` (R-Format) or from the immediate (I-Format). Then checks the forwarding path: if `forwardValid` is set and the forwarded register matches `r1` or `r2`, the stale register value is replaced with `forwardValue` and a log line is printed. Stores the decoded instruction and operand values into `ID_EX`.

void stageExecute(void)

EX stage. If `ID_EX` is invalid (bubble), clears the forwarding state and returns. Otherwise dispatches on the instruction opcode. For arithmetic/logical/shift opcodes (0–2, 5–6, 8–9): calls `execute()` to compute the result, calls `setRegister()` to write back, and sets the forwarding variables. For `MOVI` (3): writes the immediate directly to the destination register. For `BEQZ` (4): checks `val1 == 0`; if true, calls `calculateBranch()` to compute the new PC, assigns PC, and calls `flushPipeline()`. For `BR` (7): concatenates `val1` (high byte) and `val2` (low byte) into a 16-bit address, assigns PC, and calls `flushPipeline()`. For `LDR` (10) / `STR` (11): calls `readData()` or `writeData()` with the immediate as the address. Increments `completedInstructions`.

int detectHazards(void)

Output: 1 if a RAW hazard is detected, 0 otherwise

Examines whether the instruction currently in `IF_ID` (about to enter decode) reads a register that was just written by EX. Checks both `r1` and `r2` for R-Format instructions. Prints a log message when a hazard is detected. Used internally by `stageDecode()` via the forwarding check.

void flushPipeline(void)

Clears both pipeline latches (sets `valid = 0`, zeroes all fields) and resets the forwarding state. Sets `fetchCount` to the new PC value so the fetch stage correctly tracks which instruction to fetch next after the branch. Called when `BEQZ` is taken or `BR` executes.

void logState(void)

Prints the pipeline snapshot at the end of each cycle: the instruction processed in EX, the instruction waiting in ID, and the instruction waiting in IF, together with their operand values. If a forwarding path is active, also prints the forwarded register and value.

3. Hazard Handling

3.1 Data Hazards (RAW) — Forwarding

Package 3's 3-stage pipeline allows a 1-cycle window for RAW (Read-After-Write) hazards. When an instruction in EX writes a register, the very next instruction may need that value before the write-back has happened.

The implementation resolves this with full forwarding (no stalling required):

- After stageExecute() computes a result, the value and destination register are saved into forwardValue, forwardReg, forwardValid.
- During stageDecode(), if forwardValid is true and the decoded instruction reads forwardReg, the stale register value from the GPR file is replaced with forwardValue before storing into ID_EX.
- This means the correct value is always available when the instruction reaches EX — no bubbles are inserted.

3.2 Control Hazards — Flush on Branch

Branches and jumps are resolved in the EX stage. By that point, 1 or 2 younger instructions may have entered the pipeline (in ID and IF respectively). The implementation uses flush-on-branch:

- When BEQZ is taken or BR executes, flushPipeline() is called immediately.
- Both IF_ID and ID_EX latches are invalidated (valid = 0, fields zeroed).
- fetchedCount is updated to match the new PC so the fetch counter stays consistent.
- The next call to stageFetch() will fetch the correct branch-target instruction.
- The flushed instructions produce no register or memory side-effects since their valid flags are clear when they reach EX.

4. Pipeline Timing

Package 3 uses a simple 3-stage pipeline where each stage completes in exactly 1 clock cycle. There are no multi-cycle stages. The total clock cycle count for a program of n instructions (with no branches) follows the formula:

$$\text{Total Cycles} = 3 + (n - 1) \times 1$$

For 7 instructions: $3 + (6 \times 1) = 9$ clock cycles. Branches reduce the effective instruction count by flushing the 1–2 younger instructions from the pipeline.

Example pipeline diagram for 7 instructions:

Cycle	IF	ID	EX
1	Instr 1		
2	Instr 2	Instr 1	
3	Instr 3	Instr 2	Instr 1

Cycle	IF	ID	EX
4	Instr 4	Instr 3	Instr 2
5	Instr 5	Instr 4	Instr 3
6	Instr 6	Instr 5	Instr 4
7	Instr 7	Instr 6	Instr 5
8		Instr 7	Instr 6
9			Instr 7

5. Simulation Output

Per-cycle console output includes:

- A cycle-number banner.
- EX stage: instruction mnemonic, register numbers, input values, computed result, and any register or memory update.
- ID stage: instruction being decoded, register reads, and any forwarding applied.
- IF stage: PC of the fetched instruction, raw binary word, and new PC after increment.
- SREG update lines (printed by the flag-update functions in registers.c) whenever flags change.
- Pipeline snapshot from logState() at the end of each cycle.

At the end of the simulation:

- Final statistics banner (total cycles, instructions completed).
- Full GPR dump (R0–R63, decimal + hex + binary).
- SREG dump with individual flag breakdown.
- Final PC value.
- Full instruction memory dump (1024 entries, hex + binary).
- Full data memory dump (2048 entries, decimal + binary).

6. File Structure

File	Role
main.c	Entry point; initialisation and launch sequence
memory.h/c	Instruction memory, data memory, PC, parser/loader, memory print functions
registers.h/c	GPR file, SREG, flag update logic, register print functions
decoder.h/c	16-bit instruction word decoder; produces Instruction struct
alu.h/c	Arithmetic/logical/shift execution; branch address calculation; unit tests

File	Role
pipeline.h/c	3-stage pipeline controller; hazard detection; forwarding; simulation loop
sample_program.txt	Example assembly program used for testing

7. Sample Program

The repository includes `sample_program.txt` to demonstrate and test the simulator. It exercises arithmetic, logical, shift, memory, and branch instructions. To run:

```
gcc main.c memory.c registers.c decoder.c alu.c pipeline.c -o simulator
./simulator sample_program.txt
```

8. Conclusion

The Package 3 simulator implements the full specification: Harvard memory architecture, 64 general-purpose 8-bit registers, a 5-flag SREG, a 12-instruction ISA with 16-bit encoding, a 3-stage pipeline with forwarding-based RAW hazard resolution, control hazard handling via pipeline flush, and all required per-cycle and end-of-simulation console output.

The modular design — with memory, registers, decoder, ALU, and pipeline each in separate compilation units — makes the codebase easy to test, debug, and extend.